Clinic Queue Management API

1.0.0 OAS 3.0

Overview

Clinic Queue Management System (CMS) is a full-stack, multi-tenant clinic management API. It supports multiple clinics; each clinic has its own admin, doctors, receptionists, and patients. All data is **scoped by clinic** – users only see and act on data for their assigned clinic.

Student accounts: Each student receives a unique username and password that is **already linked to a specific clinic**. When implementing the frontend, students must build the UI and flows for **their assigned clinic only**, using the credentials they receive.

Student login (for frontend implementation):

- **Username:** `enrollment@darshan.ac.in`
- **Password:** `password123`

Authentication

- **Login:** `POST /auth/login` with `email` and `password` . Returns a **JWT** and user object (id, name, email, role, clinicId, clinicName, clinicCode).
- **Protected endpoints:** Send the token in the header: `Authorization: Bearer <token>` .
- **JWT payload** includes `userId` , `role` , and `clinicId` . The API uses these to enforce role-based access and clinic isolation.
- **No public registration.** Clinics and admins are created via database seed or backend script; admins create receptionists, doctors, and patients from the Admin UI. New users sign in with the email/password set by the admin.

Errors: `401` = missing or invalid token; `403` = valid token but role not allowed or user has no clinic.

Roles & Capabilities

Role	Description	Main actions
Admin	Owns one clinic.	View clinic info and counts; list users; **create** receptionists, doctors, and patients (name, email, password, role).
Patient	Belongs to one clinic.	Book appointments; view **my** appointments (with queue token/status); view appointment details with prescription and report; view **my** prescriptions and **my** reports.
Receptionist	Belongs to one clinic.	Get daily queue for a date (`date=YYYY-MM-DD`); update queue entry status (waiting → in-progress/skipped, in-progress → done).
Doctor	Belongs to one clinic.	Get **today's** queue (token, patient, appointmentId); add **prescription** (medicines, notes) and **report** (diagnosis, tests, remarks) for an appointment.

Each role can access only its allowed endpoints. Data is always filtered by the user's `clinicId` .

Key Concepts

- **Clinic:** Each clinic has a name and a unique **code** (e.g. `STUD-01`). Used for multi-tenant isolation.
- **Appointment:** Patient books a date and time slot. One appointment per slot per clinic per date. On creation, status is set to `queued` and a **queue entry** is created.
- **Queue:** One queue entry per appointment per day. Has a **token number** (per clinic per date), and status: `waiting` | `in_progress` | `done` | `skipped`. Receptionist advances status; doctor uses `appointmentId` to add prescription/report.
- **Prescription / Report:** One prescription and one report per appointment, added by the doctor (usually when status is in_progress or done).

Project Walkthrough Video

Watch the project explanation and walkthrough here: [Project walkthrough video]

Typical Workflows

1. **Admin:** Log in → Get clinic info (`GET /admin/clinic`) → List users (`GET /admin/users`) → Create doctor/receptionist/patient (`POST /admin/users`).
2. **Patient:** Log in → Book appointment (`POST /appointments`) → List my appointments (`GET /appointments/my`) → Get details with prescription/report (`GET /appointments/:id`); optionally list `/prescriptions/my` and `/reports/my`.
3. **Receptionist:** Log in → Get queue for a date (`GET /queue?date=YYYY-MM-DD`) → Update status (`PATCH /queue/:id`) as patients are called or completed.
4. **Doctor:** Log in → Get today's queue (`GET /doctor/queue`) → For each patient, add prescription (`POST /prescriptions/:appointmentId`) and/or report (`POST /reports/:appointmentId`).

Servers

Use the **Production Server** at (`https://cmsback.sampaarsh.cloud`). All paths are relative to the selected server.

Servers

https://cmsback.sampaarsh.cloud - Production (Sampaarsh)

Authorize

Auth Login (no public registration)

GET `/health` Health check

POST `/auth/login` Login

Appointments Patient: book and view appointments

POST	/appointments	Book appointment (Patient)
GET	/appointments/my	My appointments (Patient)
GET	/appointments/{id}	Appointment details with prescription and report (Patient)

Queue
Receptionist: daily queue and status updates

GET	/queue	Daily queue (Receptionist)
PATCH	/queue/{id}	Update queue status (Receptionist)

Doctor
Doctor: today's queue

GET	/doctor/queue	Today's queue (Doctor)

Prescriptions
Doctor: add; Patient: list own

GET	/prescriptions/my	My prescriptions (Patient)
POST	/prescriptions/{appointmentId}	Add prescription (Doctor)

Reports
Doctor: add; Patient: list own

GET	/reports/my	My reports (Patient)
POST	/reports/{appointmentId}	Add report (Doctor)

Admin
Admin: clinic info and create users

GET	/admin/clinic	Clinic info (Admin)
GET	/admin/users	List users (Admin)
POST	/admin/users	Create user (Admin)

Schemas

Error {

```
    error
                    string
                example: Error message
}
```

LoginRequest {

```
    email*
                    string($email)
                example: doctor1@clinic1.local
    password*
                    string
                minLength: 1
                example: password123
}
```

LoginResponse {

```
    token
                    string
                JWT for Authorization header

    user
                    {
                    id                    [...]
                    name                  [...]
                    email                 [...]
                    role                  [...]
                    clinicId              [...]
                    clinicName            [...]
                    clinicCode            [...]
                }
}
```

BookAppointmentRequest {

```
    appointmentDate*
                    string($date)
                example: 2025-03-10
                YYYY-MM-DD, today or future

    timeSlot*
                    string
                example: 10:00-10:15
                Format HH:MM-HH:MM

}
```

Appointment {

```
    id                    integer
    appointmentDate       string($date)
    timeSlot              string
    status                string
                          Enum:
                          [ scheduled, queued, in_progress, done, cancelled ]
    patientId             integer
    clinicId              integer
    createdAt             string($date-time)
    queueEntry
```

QueueEntry {

```
    id                    integer
    tokenNumber           integer
    status                string
                          Enum:
                          [ waiting, in_progress, done, skipped ]
    queueDate             string($date)
    appointmentId         integer
    appointment           {
                          patient
                                {
                                name
                                      {
                                      }
                                phone
                                      {
                                      }
                                }
                          }
                          }
}
```

QueueEntry

```
{
    id                          integer
    tokenNumber                 integer
    status                      string
                              Enum:
                                [ waiting, in_progress, done, skipped ]
    queueDate                   string($date)
    appointmentId               integer
    appointment                 {
                               patient
                                          {
                                          name
                                                                  {
                                                                }
                                          phone
                                                                  {
                                                                }
                                        }
                              }
}
```

QueueUpdateRequest

```
{
    status*                     string
                              waiting->in-progress|skipped; in_progress->done
                              Enum:
                                [ in-progress, done, skipped ]
}
```

DoctorQueueItem

```
{
    id                          integer
    tokenNumber                 integer
    status                      string
    patientName                 string
    patientId                   integer
    appointmentId               integer
}
```

PrescriptionMedicine

```
{
    name*                       string
                              example: Paracetamol
    dosage*                     string
                              example: 500mg
    duration*                   string
                              example: 5 days
}
```

AddPrescriptionRequest {

```
    medicines*
                    [
                minItems: 1
                PrescriptionMedicine {
                    name*
                        string
                    example: Paracetamol
                    dosage*
                        string
                    example: 500mg
                    duration*
                        string
                    example: 5 days
                }]
    notes
                    string
                example: After food
}
```

AddReportRequest {

```
    diagnosis*
                    string
                example: Viral Fever
    testRecommended
                    string
                example: Blood Test
    remarks
                    string
                example: Rest for 3 days
}
```

AdminCreateUserRequest {

```
    name*
                    string
                minLength: 3
                example: John Doe
    email*
                    string($email)
    password*
                    string
                minLength: 6
    role*
                    string
                Enum:
                    [ receptionist, patient, doctor ]
    phone
                    string
}
```